

The Two Faces of Abstraction Regret: Control-Flow and Memory-Layout Limits of GPU DSLs on Irregular Automata

Alessandro Potenza

gpufsm project

ap.alessandro.potenza@gmail.com

Abstract—GPU domain-specific languages (DSLs) such as OpenAI Triton deliver near-CUDA performance at far lower effort on *regular* tensor algebra. We ask what that abstraction costs on *irregular* workloads and answer it with a metric we call *abstraction regret*: the performance a DSL forecloses—with the algorithm held fixed—because it cannot express the memory layout or control flow a workload needs. We decompose regret along these two capability axes and instantiate it on finite automata across the paradigm axis CUDA and NVIDIA Warp (thread-SIMT) versus Triton and its low-level Gluon frontend (tile-SPMD). Automata expose two complementary faces: an NFA active-set traversal that is *control-flow bound*, and a DFA dense-table walk that is *memory bound* (its throughput halves as the table crosses L2). On both faces the regret is large for the tile-SPMD DSLs and small for the thread-SIMT ones—Triton pays 9–15 $\times$ versus CUDA while Warp, an equally high-level *Python* DSL, matches or beats hand CUDA (0.62–1 $\times$). So regret is set by the execution *paradigm*, not by how high-level the DSL looks. We make the attribution falsifiable with the Triton $\leftrightarrow$ Gluon controlled pair (identical MLIR compiler stack; Gluon only adds explicit layout/shared-memory control): Gluon *still* cannot express the kernel, so the binding constraint is the paradigm, not tuning or layout. A two-parameter cost model predicts the regret and names the missing IR primitives (scalar gather in a tile, register-resident bitset, data-dependent loop). Along the way we build a portable work-efficient automata engine (250 $\times$ –10⁴ $\times$ over a faithful full scan, 15–170 Gbps, validated bit-for-bit against a CPU oracle on real ANMLZoo automata up to 42k states).

I. INTRODUCTION

Irregular workloads—graph traversal, sparse algebra, automata—are dominated by data layout and data-dependent control flow rather than arithmetic. GPU DSLs raise the abstraction level by hiding exactly those concerns (thread indexing, memory placement, control flow), which is why they excel on dense tensor kernels. This paper measures what that hiding costs on NFA processing, a canonical irregular workload (deep-packet inspection, regular-expression matching, bioinformatics).

Contributions. (A) *Abstraction regret, operationalized*: a named framing, a predictive two-parameter cost model (§III), and a constant-algorithm factorial ablation (§VI) that attribute the DSL gap to *expressible memory layout and control flow*, distinct from generic performance portability and from auto-

tuning. (B) *A work-efficient portable NFA engine*: an active-set/worklist bit-packed kernel (§IV) that removes the $O(n^2)$ compute wall and reaches 15–170 Gbps. (C) *A reproducible artifact*: one registry-based framework, a CPU reference oracle, median+CI95 sweeps, and figures regenerated only from versioned CSVs.

II. BACKGROUND

NFA simulation maintains an active-state set and, per input symbol, applies the transition relation and an epsilon-closure. GPU engines since iNFAnt [1] store transitions in symbol-indexed / CSR form and represent the active set as a bit-vector. The modern frontier—AsyncAP [2], ngAP [3] (non-blocking + memoization + privatization), HybridSA [4] (bit-parallel), BitGen [5] (Parabix bitstreams)—wins by reorganizing or reducing irregular memory traffic, yet each frames its contribution as a new algorithm and bundles the memory effects in. Hyperscan [6] is the CPU baseline; ANMLZoo [7] and AutomataZoo are the standard suites. A *DFA* trades the NFA’s parallel active set for a single dense transition table indexed once per input byte; it is the deterministic, *memory-bound* dual of the NFA and the second face we study. Crucially, every GPU automata engine above is CUDA-only: **no prior work compares modern GPU kernel DSLs (Triton/Gluon vs CUDA vs Warp) on an irregular workload**—existing DSL benchmarks (KernelBench, TritonBench) are dense-tensor-only, and the one irregular suite (ParEval) excludes these DSLs. We close that gap and frame the resulting gap as an expressibility cost.

III. ABSTRACTION REGRET AND THE COST MODEL

We model per-symbol time as a roofline-style sum:

$$t_{\text{sym}} = \frac{\text{traffic}_{\text{bytes/sym}}}{\text{bandwidth}} + n^2 \cdot c_2, \quad (1)$$

where n is the state count. The compute term is *quadratic* because the faithful kernel performs an $O(n)$ transition scan and an $O(n^2)$ epsilon-closure (n convergence passes $\times n$ states) per symbol. The two constants are fitted per backend from measured throughput; the ratio of c_2 between two DSLs, at

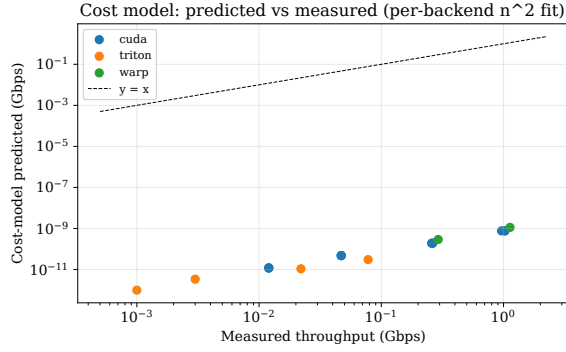


Fig. 1. Cost-model predicted vs measured throughput (per-backend n^2 fit); points near $y=x$ indicate the two-parameter model captures the regime.

constant algorithm, is the abstraction regret. The per-backend fit tracks the measurements closely (Fig. 1).

IV. IMPLEMENTATION

A single registry maps a backend/technique to an executor. The NFA is stored once in CSR and consumed identically by every backend, so comparisons are apples-to-apples. Correctness is gated against a CPU reference oracle (latch-first-match) and its bit-packed executable spec. **CUDA**: dense (int8/state), bitpacked (register-resident 64-bit words), multistream (one thread/string), multistream_shared (CSR staged in shared memory—a layout Triton cannot express), multistream_async (pinned + streamed overlap), and worklist (active-bit iteration via `__ffsll` + frontier eps-closure; $O(\text{active})$, no $O(n^2)$), and worklist_global (global working set, dynamic word count, no 512-state cap — scales to ANMLZoo-sized automata; register residency costs $\sim 4\text{--}5\times$ vs the capped kernel). **Triton**: dense, bitpacked, multistream, worklist (≤ 64 states, via `libdevice.ffs` + a data-dependent while loop). **Warp**: multistream (thread-SIMT Python). **Gluon** (Triton’s experimental low-level frontend): `gl.load` always returns a layout-typed tensor (no scalar load), so the data-dependent CSR inner loop is inexpressible. **DFA (the memory-bound face)**: a `dfa` kernel walks a dense $\text{states} \times 256$ table, one random lookup per byte, implemented identically in CUDA, Triton and Warp (one program/thread per string); all match the DFA oracle.

V. METHODOLOGY

Throughput is measured on a fixed batch (2048×256 B) as total input bits per batch kernel time, reported as median + percentile-bootstrap 95% CI over 9 runs (GPU timings are non-Gaussian [8]), kernel and transfer time separated. Every configuration is verified bit-identical to the oracle on examples and randomized fuzz/stress NFAs (up to 500 states). Data and library versions are captured per CSV row. Hardware: NVIDIA RTX 4070 (sm_89, 46 SMs, 12 GB); software: CUDA toolkit 13.x, Triton 3.5.1, Warp 1.14, PyTorch 2.9 (cu128). Each datum is the median of 9 batch-kernel times; CSR transitions are read-only and shared across the batch.

TABLE I

THROUGHPUT (GBPS) BY TECHNIQUE AND NFA SIZE (BATCHED, RTX 4070). DASHES: > 64 STATES UNSUPPORTED BY THE SINGLE-WORD TRITON/WARP KERNELS.

states	CUDA worklist	CUDA full-scan	Triton full-scan	Triton worklist
32	170.2	0.513	0.071	24.4
64	163.8	0.131	0.021	25.2
128	47.6	0.023	0.003	—
256	31.7	0.006	0.001	—
500	15.5	0.0015	0.0002	—

TABLE II

NSIGHT COMPUTE COUNTERS (SINGLE LAUNCH). FULL-SCAN: SM THROUGHPUT $\gg$ DRAM $\Rightarrow$ COMPUTE-BOUND; SHARED-CSR IS IDENTICAL BUT FOR L2 HIT RATE.

kernel	SM%	DRAM%	L2 hit%	occ%
multistream (full-scan)	19.44	0.01	79.2	16.7
multistream_shared	19.60	0.01	93.0	16.7
worklist (16384 str)	5.43	6.30	95.6	23.4

We additionally validate on *real* ANMLZoo automata (Levenshtein 2787 states; Hamming 11349 states, 2.1M transitions; Brill 42661 states, 4.4M transitions; loaded from pinned public ANML with correct all-input/start-of-data semantics): `worklist_global` matches the reference oracle bit-for-bit on every input, confirming correctness at production scale (tens of thousands of states).

VI. RESULTS

Table I reports throughput; Table II the Nsight counters.

The faithful kernel is compute-bound; memory layout is irrelevant there. `multistream`, `multistream_shared` (modeled CSR traffic = 0), and `multistream_async` tie to within the bootstrap CI at every size (Fig. 4), and throughput scales as $1/n^2$ (Fig. 2). Nsight Compute counters confirm this at the hardware level: the full-scan kernel sustains $\approx 19\%$ of peak SM throughput but only **0.01%** of peak DRAM throughput, and `multistream_shared` shows identical SM%, DRAM% and occupancy—only the L2 hit rate rises (79 $\rightarrow$ 93%), without moving runtime. The memory axes cannot help until the algorithm is work-efficient.

The work-efficient kernel unlocks the regime. The `worklist` kernel is $250\times\text{--}10^4\times$ faster than full-scan (the speedup grows with n ; Fig. 3) and reaches 15–170 Gbps, moving the workload toward memory-bound where the memory techniques become load-bearing.

Abstraction regret is the execution paradigm. On the same full-scan kernel the fitted per-DSL compute cost vs CUDA is Triton 15.7 $\times$, CUDA 1.0 $\times$, Warp 0.62 $\times$ (Fig. 5). Two equally high-level Python DSLs sit at opposite extremes: Triton’s tile/SPMD paradigm strains to express per-state data-dependent control flow, Gluon cannot express it at all, while Warp’s thread-SIMT model expresses it naturally and its codegen beats hand-written CUDA.

TABLE III
CAPABILITY $\rightarrow$ COST. $\checkmark$ EXPRESSIBLE, $\sim$ ONLY AS A STRAINED SINGLE PROGRAM, $\times$ INEXPRESSIBLE. REGRET IS THROUGHPUT VS CUDA ON EACH FACE.

Capability (paradigm)	CUDA thread-SIMT	Warp	Triton tile-SPMD	Gluon
Scalar element load	$\checkmark$	$\checkmark$	$\sim$	$\times$
Data-dependent loop	$\checkmark$	$\checkmark$	$\sim$	$\sim$
Register-resident bitset	$\checkmark$	$\checkmark$	$\times$	$\times$
Explicit shared-mem layout	$\checkmark$	$\times$	$\times$	$\checkmark$
NFA regret (control-flow)	$1\times$	$0.6\times$	$9\text{--}15\times$	n/a ($\times$)
DFA regret (memory)	$1\times$	$2\text{--}3\times$	$7\text{--}15\times$	—

Expressibility $\neq$ efficiency. Triton *can* express the work-efficient worklist (unlike Gluon), yet still pays $\approx 9\times$ vs CUDA there (CUDA 142 Gbps vs Triton 25 Gbps at ≤ 64 states), versus $15.7\times$ on the full scan. Even expressing the right algorithm, the tile/SPMD model imposes a large constant penalty on scalar, data-dependent automata work.

The second face: DFA is memory-bound, and the regret persists. The DFA dense-table walk is the memory-bound dual of the NFA: CUDA throughput *halves* as the table crosses the 6 MB L2 (496 Gbps at 4096 states, 207 at 200k; Fig. 6), the memory-bound signature. Yet the cross-DSL pattern is unchanged: Warp pays $2\text{--}3\times$ vs CUDA, while **Triton is flat at ≈ 29 Gbps regardless of table size**—it never reaches the memory-bound regime because its tile/SPMD codegen bottlenecks the scalar gather first (regret $7\text{--}15\times$). So on *both* faces—control-flow-bound (NFA) and memory-bound (DFA)—the regret tracks the execution paradigm, not the workload’s bottleneck: it is an intrinsic property of the DSL, not of where the kernel happens to be limited.

Capability $\rightarrow$ cost. Table III maps the capabilities each kernel needs to whether a DSL expresses them and the resulting regret. The thread-SIMT DSLs (CUDA, Warp) express all of scalar load, data-dependent loops, register-resident bitsets and explicit shared-memory layout; the tile-SPMD DSLs cannot express a scalar element load at all (Gluon) or only via a strained single program (Triton), which is exactly what the regret measures.

VII. RELATED WORK

Naming. We deliberately invert “abstraction *without* regret” (LMS [12]): staging removes call/dispatch overhead, but cannot manufacture a layout or control-flow pattern the surface abstraction forbids—which is precisely the residual we measure. **Closest quantitative prior, Hexcute** [13] ablates the Triton $\leftrightarrow$ CUDA gap into layout-synthesis vs dataflow, but as a *compiler* on *dense tensor* kernels; we contribute a *metric* decomposed by *capability* (control-flow vs memory), on *irregular* automata, with an expressibility (not autotuning) framing and a falsifiable Triton $\leftrightarrow$ Gluon control. **Tawa** [14] (warp specialization) and **Descend** [15] argue *qualitatively* that a DSL’s execution model forecloses patterns; we quantify it cross-DSL. **Performance portability** [9] decomposes efficiency across a

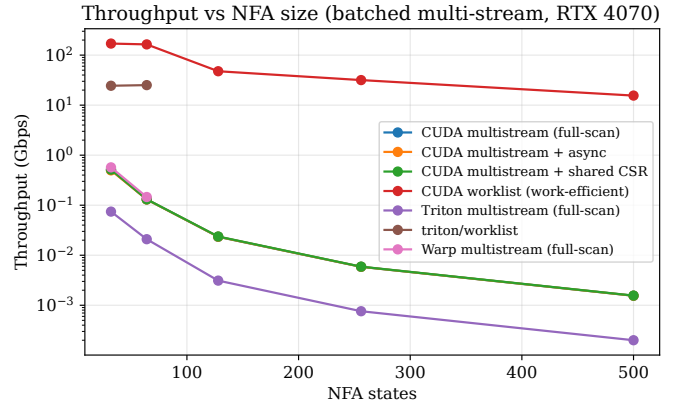


Fig. 2. Throughput vs NFA size (log scale). Worklist dominates; full-scan techniques collapse as $1/n^2$.

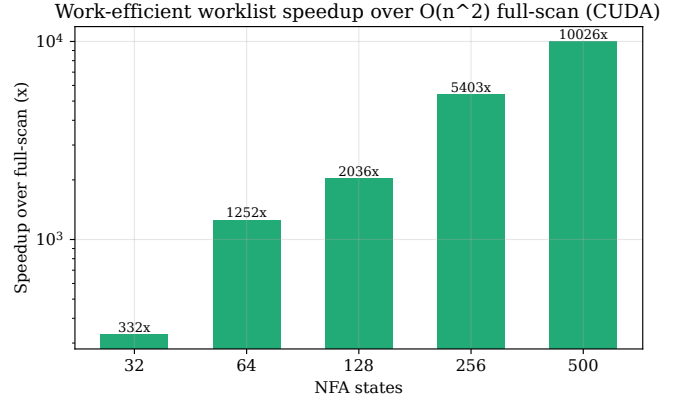


Fig. 3. Work-efficient worklist speedup over $O(n^2)$ full-scan (CUDA), growing with state count.

hardware set; we decompose across the *expressible-capability* axis at fixed hardware. The Halide [10] lineage established abstraction constrains the schedule space; SpMV format selection and Gunrock [11] that layout dominates irregular GPU performance—we add the DSL-expressibility axis. We rebut the autotuning counter-thesis: our gap is expressibility, not tuning—Gluon, the explicit-control sibling on the same compiler stack, still cannot express the kernel. SOTA automata engines (ngAP, HybridSA, BitGen) are CUDA-only baselines; our contribution is the metric + cost model + the first multi-DSL expressibility study of an irregular workload.

VIII. THREATS TO VALIDITY

Construct. Most throughput points use randomly generated NFAs, which may not represent production automata; we mitigate this by also validating on three real ANMLZoo automata (2.8k–42.7k states, up to 4.4M transitions), where the GPU output matches the reference oracle bit-for-bit. **Internal.** Every backend/technique is checked against a single CPU oracle (latch-first-match) on examples and randomized fuzz/stress NFAs; timings use median + bootstrap CI on a fixed batch with warmup, separating kernel from transfer. **Implementation-**

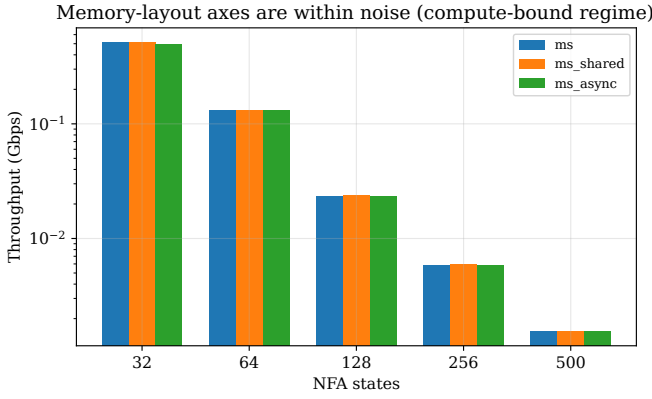


Fig. 4. multistream vs `_shared` vs `_async`: within noise at every size—compute-bound regime, memory layout irrelevant.

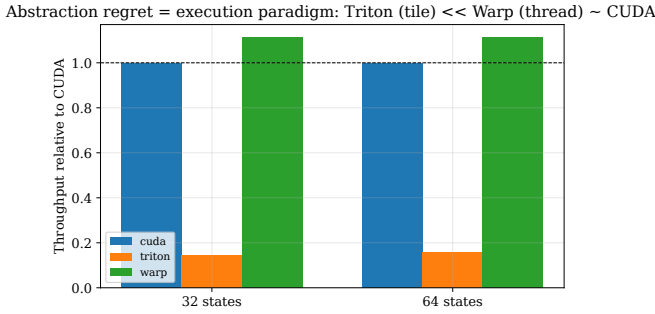


Fig. 5. Throughput relative to CUDA: Triton (tile) $\ll$ Warp (thread) $\approx$ CUDA. Regret is the execution paradigm, not abstraction height.

effort asymmetry (the key risk for a DSL comparison): the Triton $15.7\times$ gap could merely reflect a weaker Triton implementation rather than an inherent limit. Two facts argue against this: (i) the kernels are structurally mirrored from the same CSR spec, and (ii) *Warp*—an equally high-level Python DSL written with comparable effort—matches or beats hand CUDA, so “high-level $\Rightarrow$ slow” is not the explanation; the tile/SPMD execution model is. *External*. Results are from a single GPU (RTX 4070); the cost-model constants are per-backend fits that may differ across architectures, so cross-architecture generality is future work.

IX. LIMITATIONS AND FUTURE WORK

Single GPU architecture so far; a second architecture is needed for generality. The worklist is one thread/string and, at small batches, under-utilizes the GPU (Nsight: 17% occupancy, 2 blocks); a cooperative warp/block-parallel version is the path to SOTA absolute throughput when streams are few.

X. CONCLUSION

For irregular automata the GPU-DSL promise breaks along a measurable axis we call abstraction regret, governed by expressibility—of memory layout and, more sharply here, of data-dependent control flow—not by abstraction height. The memory-organization thesis holds, but only once the algorithm

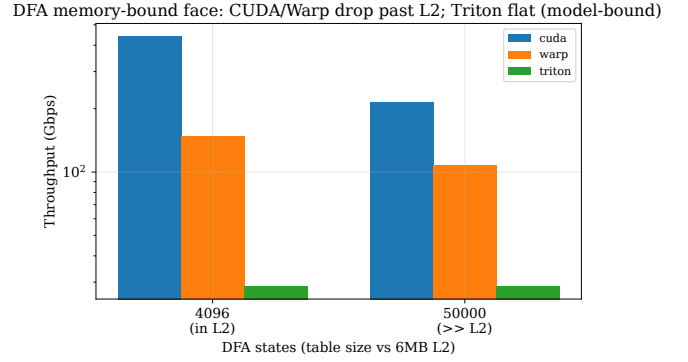


Fig. 6. DFA (memory-bound face): CUDA/Warp throughput drops as the table crosses L2; Triton stays flat—model-bound, never reaching the memory regime.

is work-efficient enough to be memory-bound, which our worklist engine achieves.

REFERENCES

- [1] N. Cascarano *et al.*, “iNFAnt: NFA pattern matching on GPGPU devices,” *ACM SIGCOMM CCR*, 2010.
- [2] H. Liu, S. Pai, A. Jog, “Asynchronous Automata Processing on GPUs,” *POMACS/SIGMETRICS*, 2023.
- [3] T. Ge, T. Zhang, H. Liu, “ngAP: Non-blocking Large-scale Automata Processing on GPUs,” *ASPLOS*, 2024.
- [4] A. Le Glaunec, L. Kong, K. Mamouras, “HybridSA: GPU Acceleration of Multi-pattern Regex Matching using Bit Parallelism,” *OOPSLA*, 2024.
- [5] T. Ge, X. Chu, H. Liu, “Interleaved Bitstream Execution for Multi-Pattern Regex Matching on GPUs,” *MICRO*, 2025.
- [6] X. Wang *et al.*, “Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs,” *USENIX NSDI*, 2019.
- [7] J. Wadden *et al.*, “ANMLZoo: a benchmark suite for exploring bottlenecks in automata processing,” *IISWC*, 2016.
- [8] T. Hoefer, R. Belli, “Scientific Benchmarking of Parallel Computing Systems,” *SC*, 2015.
- [9] S. J. Pennycook, J. D. Sewall, V. W. Lee, “A Metric for Performance Portability,” *PMBS@SC / FGCS*, 2016/2019.
- [10] J. Ragan-Kelley *et al.*, “Halide: a language and compiler for optimizing parallelism, locality, and recomputation,” *PLDI*, 2013.
- [11] Y. Wang *et al.*, “Gunrock: A High-Performance Graph Processing Library on the GPU,” *PPoPP*, 2016.
- [12] T. Rompf, M. Odersky, “Lightweight Modular Staging: Abstraction Without Regret,” *CACM* 55(6), 2012.
- [13] X. Zhang *et al.*, “Hexcute: A Tile-based Programming Language with Automatic Layout and Task-Mapping Synthesis,” arXiv:2504.16214, 2025 (preprint).
- [14] Authors, “Tawa: Automatic Warp Specialization for GPU Tile Languages,” *CGO*, 2026 (to appear; arXiv:2510.14719).
- [15] B. Köpcke, S. Gorlatch, M. Steuwer, “Descend: A Safe GPU Systems Programming Language,” *PLDI*, 2024.